C Programming 101 — Practice Test (With Answers)

Instructions

24 multiple-choice questions (3 points each) and one short program (28 points), 100 points total.

- Circle one answer per multiple-choice question.
- Assume a 64-bit Linux machine and `gcc -Wall -Wextra -Werror -std=c11`, as in lab.
- Every code snippet is inside a working `main` with `#include <stdio.h>` unless shown otherwise.

Section	Questions	Points
1 — Compiling and Linking	1–8	24
2 — Data Types	9–16	24
3 — Control Statements	17–24	24
4 — Small Program	25	28
Total		**100**

Section 1 — Compiling and Linking (24 points)

1. In what order does `gcc` run the four stages of building a program?

A) Compile, preprocess, link, assemble
B) Preprocess, compile, assemble, link
C) Preprocess, assemble, compile, link
D) Link, preprocess, compile, assemble

2. What does `gcc -c main.c` produce?

A) An executable named `a.out`
B) An assembly file `main.s`
C) An object file `main.o`
D) A preprocessed file `main.i`

3. Which stage handles `#include` and `#define`?

A) Preprocessor
B) Compiler
C) Assembler
D) Linker

4. A build fails with `undefined reference to 'greet'`. Which stage reported this error?

A) Preprocessor
B) Compiler
C) Assembler
D) Linker

5. `main.c` calls `greet()` but the compiler warns `implicit declaration of function 'greet'`. What is the most likely cause?

A) `main.c` has no prototype for `greet`; the header is missing or not included
B) `greet.o` was left off the link command
C) The Makefile is missing a `clean` target
D) `greet` returns `void`

6. Which command links `main.o` and `greet.o` into an executable named `hello`?

A) `gcc -c main.o greet.o -o hello`
B) `gcc main.o greet.o -o hello`
C) `gcc -E main.o greet.o -o hello`
D) `gcc -S main.c greet.c -o hello`

7. Given this Makefile line, what is the value of `OBJS`?

```
SRCS = main.c greet.c
OBJS = $(SRCS:.c=.o)
```

A) `main.c greet.c`
B) `main.o greet.o`
C) `main greet`
D) `hello`

8. You run `make`, then edit only `greet.c` and run `make` again. What happens?

A) Every `.c` file is recompiled
B) Only `greet.c` is recompiled, then the program is relinked
C) Only `main.c` is recompiled
D) Nothing; `make` only builds once

Section 2 — Data Types (24 points)

9. What does this print?

```c
printf("%d\n", 7 / 2);
```

A) `3.5`
B) `3`
C) `4`
D) `3.0`

10. What does this print?

```c
double x = 7 / 2;
printf("%.1f\n", x);
```

A) `3.5`
B) `3.0`
C) `4.0`
D) It does not compile

11. What does this print?

```c
double x = (double)7 / 2;
printf("%.1f\n", x);
```

A) `3.5`
B) `3.0`
C) `4.0`
D) `3`

12. Which statement correctly reads a `double` named `x` from the keyboard?

A) `scanf("%f", &x);`
B) `scanf("%lf", &x);`
C) `scanf("%d", &x);`
D) `scanf("%lf", x);`

13. What does this print?

```c
char c = 'A';
printf("%d\n", c);
```

A) `A`
B) `65`
C) `a`
D) `0`

14. Which declares an integer whose value cannot be changed after it is set?

A) `static int max = 10;`
B) `const int max = 10;`
C) `int max == 10;`
D) `fixed int max = 10;`

15. What is the value of `x` after this runs?

```c
int x = 17;
x = x % 5;
```

A) `3`
B) `3.4`
C) `2`
D) `5`

16. On the 64-bit Linux lab machines, what does `sizeof(int)` return?

A) 1
B) 2
C) 4
D) 8

Section 3 — Control Statements (24 points)

17. What does this print?

```
int x = 10;
if (x > 5)
    printf("A");
else if (x > 8)
    printf("B");
else
    printf("C");
```

A) A
B) B
C) AB
D) C

18. What does this print? (Assume warnings are not treated as errors.)

```
int x = 0;
if (x = 5)
    printf("yes");
else
    printf("no");
```

A) no
B) yes
C) Nothing
D) yesno

19. What does this print?

```
int n = 2;
switch (n) {
    case 1:  printf("one ");
    case 2:  printf("two ");
    case 3:  printf("three ");
             break;
    default: printf("other");
}
```

A) two
B) two three
C) two three other
D) one two three

20. What does this print?

```c
for (int i = 0; i < 5; i++)
    printf("%d ", i);
```

A) `1 2 3 4 5`
B) `0 1 2 3 4 5`
C) `0 1 2 3 4`
D) `1 2 3 4`

21. How many times does the body of this loop run?

```c
int i = 10;
do {
    printf("hi\n");
} while (i < 5);
```

A) 0
B) 1
C) 5
D) It never stops

22. What does this print?

```c
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0)
        continue;
    if (i > 7)
        break;
    printf("%d ", i);
}
```

A) `1 3 5 7`
B) `1 3 5 7 9`
C) `2 4 6`
D) `1 2 3 4 5 6 7`

23. What happens when this runs?

```c
int i = 0;
while (i < 3) {
    printf("%d ", i);
}
```

A) Prints `0 1 2`
B) Prints `0 1 2 3`
C) Prints `0` forever; the loop never ends
D) Prints nothing

24. What is the value of `count` after this runs?

```c
int count = 0;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        count++;
    }
}
```

A) 5
B) 6
C) 3
D) 2

Section 4 — Small Program (28 points)

25. Write a complete C program that does the following:

1. Prints the prompt `Enter a number:` and reads an `int` named `n`.
2. If `n` is less than 1, prints `Invalid` and ends.
3. Otherwise, for every number from 1 to `n`, prints the number followed by `even` or `odd`, one per line.
4. Prints the sum of 1 through `n` as `Sum: <sum>`.
5. Prints the average as `Average: <average>` with exactly two decimal places. The average must be a `double`, not integer division.

Sample run (the user types `4`):

```
Enter a number: 4
1 odd
2 even
3 odd
4 even
Sum: 10
Average: 2.50
```

Sample run (the user types `0`):

```
Enter a number: 0
Invalid
```

Requirement	Points
Compiles with `-Wall -Wextra -Werror -std=c11`	4
Prompts and reads `n` with `scanf`	4
Rejects `n` less than 1 with `if`	4
Loop from 1 to `n` with correct `even` / `odd`	8
Correct sum	3
Average as a `double`, printed with	5

Requirement	Points
`%.2f`	
Total	**28**

Instructor Answer Key

All outputs below were verified by compiling and running the snippets with gcc.

Q	Answer	Why
1	B	gcc runs preprocess, compile, assemble, link, in that order.
2	C	`-c` stops after the assemble stage and keeps the `.o` file.
3	A	`#include` and `#define` are preprocessor directives.
4	D	"undefined reference" means the linker could not find the function's code.
5	A	The compiler saw a call with no prototype; include the header.
6	B	No `-c`, `-E` or `-S`, so gcc links the objects into `hello`.
7	B	`$(SRCS:.c=.o)` swaps each `.c` suffix for `.o`.
8	B	Make compares timestamps; only `greet.o` is out of date, then `hello` relinks.
9	B	`int / int` is integer division; the fraction is dropped.
10	B	`7 / 2` is computed as `int` (3) before it is stored in the `double`.
11	A	The cast makes one operand a `double`, so the division keeps the fraction.
12	B	`scanf` needs `%lf` for `double` and the address `&x`.
13	B	A `char` is a small integer; `'A'` is 65 in ASCII.
14	B	`const` makes the variable read-only after initialization.
15	C	17 divided by 5 leaves a remainder of 2.
16	C	`int` is 4 bytes on 64-bit Linux.
17	A	The first true condition wins; the rest of the chain is skipped.
18	B	`x = 5` assigns 5, which is nonzero, so it is true. With `-Wall -Werror` this would not compile.
19	B	No `break` after case 2, so it falls through to case 3, which breaks.
20	C	Starts at 0 and stops before 5.
21	B	A `do-while` checks its condition after the body, so the body runs once.
22	A	Even numbers are skipped; at 9 the `break` ends the loop.
23	C	`i` is never incremented, so `i < 3` is always true.
24	B	The inner loop runs 2 times for each of 3 outer passes: 3 × 2 = 6.

Question 25 model solution

Compiles cleanly with `-Wall -Wextra -Werror -std=c11` and produces both sample runs exactly.

```c
#include <stdio.h>

int main(void)
{
    int n;
```

```c
    int sum = 0;

    printf("Enter a number: ");
    scanf("%d", &n);

    if (n < 1) {
        printf("Invalid\n");
        return 0;
    }

    for (int i = 1; i <= n; i++) {
        if (i % 2 == 0)
            printf("%d even\n", i);
        else
            printf("%d odd\n", i);
        sum += i;
    }

    double average = (double)sum / n;
    printf("Sum: %d\n", sum);
    printf("Average: %.2f\n", average);
    return 0;
}
```

Common mistake to watch for: `double average = sum / n;` prints `2.00` instead of `2.50` for an input of 4.

Grading question 25 with grade.sh

`input.txt`

```
Enter a number:=4
```

`expected.txt`

```
1 odd
2 even
3 odd
4 even
Sum: 10
Average: 2.50
```